

GREENLIGHT

Ultimate Claude Code Build Brief

Personal Sustainability Negotiator

Search is not action. A rebate finder gives you a link. Greenlight gives you an agent.

CentennialHacks • Sustainability & Innovation • MLH Best Use of Gemini

BUILD PRINCIPLES

- Money first. Carbon second.
- Evidence over AI confidence.
- Search is not action.
- AI interprets. Code calculates.
- Verified routing before communication.
- Human approval at consequential boundaries.
- Desktop and mobile are both first-class experiences.

BUILD BRIEF: GREENLIGHT

You are acting as the founding engineer, senior product designer, AI systems architect, backend engineer, frontend motion designer, and hackathon execution lead for this project.

I need you to BUILD the application end-to-end, not merely describe it.

Inspect the repository first, understand what already exists, then implement the strongest possible version of this product.

You have design freedom, but you must preserve the product thesis, agent behavior, UX, technical architecture, trust model, mobile/desktop responsiveness, and demo choreography described below.

Do not stop at scaffolding.

Do not return a generic implementation plan instead of code.

Do not turn this into a chatbot.

0. PRODUCT IN ONE SENTENCE

Build an AI agent that ingests something as mundane as a household utility bill, understands the user's circumstances, discovers relevant sustainability incentives and financially beneficial efficiency actions, determines what the user may qualify for, calculates the economics, identifies the exact institution or program administrator responsible for the next step, prepares the necessary application data or communication, and moves the user as close to completion as safely possible.

The core transformation is:

messy personal information -> eligibility intelligence -> financial optimization -> verified action routing -> agentic execution

1. WHAT THIS IS NOT

This is NOT:

- a carbon footprint dashboard
- a generic rebate finder
- an ESG dashboard
- a climate chatbot
- a list of green tips
- an eco-score
- a guilt-based sustainability app

- a government links directory
- a static recommendation engine

If the experience ever becomes:

"Here are some programs you might qualify for."

and stops there, the product has failed.

The product must continue toward action.

2. PRODUCT THESIS

The climate transition has a UX problem.

Governments, utilities, municipalities, manufacturers, and other institutions already offer:

- rebates
- tax credits
- energy programs
- financing
- retrofit incentives
- efficiency programs
- utility discounts
- electrification incentives

The problem is not merely that people do not know these programs exist.

The real problem is that acting on them often means navigating:

- government websites
- utility websites
- PDFs
- eligibility rules
- technical language
- application portals
- documentation requirements
- contractor requirements
- receipts
- quotes
- deadlines
- forms
- contact channels
- departments
- service organizations

The user frequently has to become an expert in bureaucracy just to save money.

That is backwards.

Core statements:

Search is not action. A rebate finder gives you a link. Greenlight gives you an agent.

AI interprets. Evidence decides.

Stop asking people to become sustainability experts. Give them an agent that understands the system for them.

If someone is willing to pay you to go green, your AI should make sure you collect.

3. PRIMARY MOTIVATOR

Do not lead with guilt.

Avoid:

- "your footprint is too high"
- "you emitted X tonnes"
- environmental shame
- eco morality scores
- gamified guilt
- excessive leaf imagery

Lead with money.

Instead of:

Your household generated X tonnes of CO2.

Say:

We found \$2,840 you may be leaving on the table.

Instead of:

You should install a heat pump.

Say:

This upgrade may qualify for \$X in incentives, with an estimated net cost of \$Y and a payback period of Z years.

Sustainability should become financially legible.

4. WHY IT IS CALLED A NEGOTIATOR

The product should support user objectives such as:

Save me \$1,000 this year without spending more than \$500 upfront.

Find every incentive I may already qualify for.

Reduce my energy bill without replacing major equipment.

Give me the lowest upfront-cost path.

Reduce household emissions by 20% without increasing annual expenses.

Greenlight should optimize among competing options and constraints.

It is not merely recommending.

It is negotiating the best realistic path for the user.

5. HACKATHON GOAL

This is for CentennialHacks.

Primary challenge:

Sustainability & Innovation

Secondary target:

MLH Best Use of Gemini

The demo should make sense within roughly 2-4 minutes.

Optimize for:

1. immediate comprehension
2. visible transformation
3. genuinely useful Gemini integration
4. credible real-world value
5. beautiful UX
6. visible agentic action
7. evidence and trust
8. deterministic demo reliability
9. polished desktop experience
10. equally functional mobile experience

The app should look like a startup product, not a hackathon template.

6. PRODUCT NAME

Use:

GREENLIGHT

Supporting line:

make the greener move the obvious move.

Internal description:

Your personal sustainability negotiator.

Make the product name and tagline configurable from one central file.

7. CRITICAL UX RULE

DO NOT BUILD A CHATBOT AS THE MAIN INTERFACE

There may be a small conversational command field for goals, but the main experience should be:

upload -> understand -> discover -> compare -> route -> act

The interface should make the agent's work visible through:

- analysis states
- evidence rows
- opportunity scoring
- timelines
- progress
- contact resolution

- application preparation
- approval checkpoints

The user should feel:

"This thing is actually doing administrative work for me."

not:

"I'm talking to another AI assistant."

8. PRIMARY DEMO STORY

Scene 1 - Landing

Hero:

What are you leaving on the table?

Body:

Upload a utility bill. Greenlight finds relevant incentives, calculates the economics, figures out who actually handles the next step, and prepares the work required to pursue it.

Primary CTA:

Upload a bill

Secondary CTA:

Try a demo household

Accepted demo formats:

- PDF
- PNG
- JPG

Small trust note:

We never submit anything or contact anyone without your approval.

9. SCENE 2 - BILL INGESTION

When a bill is uploaded, do not immediately jump to results.

Create a fast cinematic analysis sequence.

Example:

Reading document

Then:

Toronto, Ontario

Residential electricity account

Utility provider detected

Estimated annualized consumption: 8,420 kWh

Billing structure detected

Potential efficiency opportunity detected

Then:

Building household profile...

Then:

Searching programs...

Then:

Resolving administrators and required actions...

Target:

2-5 seconds live.

2-3 seconds deterministic in demo mode.

Do not use fake long "AI thinking" animations.

10. SCENE 3 - VALUE REVEAL

Transform the screen.

Large central value:

\$3,240

Supporting:

potential value found

Then:

4 opportunities appear worth pursuing.

Example opportunity cards:

Smart thermostat

Potential incentive

\$100

Eligibility confidence

96%

Effort

Low

Estimated annual savings

\$74

Next step

Program application

Status

Ready to pursue

Heat pump upgrade

Potential incentive

Up to \$X

Eligibility confidence

78%

Estimated net upfront cost

\$X

Estimated payback

X.X years

Next step

Verify property criteria

Status

Needs 2 answers

11. IMPORTANT LANGUAGE RULE

Never fabricate eligibility.

Use:

- Potential incentive
- Likely eligible
- Appears compatible
- Estimated eligibility
- Verification required
- Program requirements appear to match

Never use:

- Guaranteed
- Approved
- You will receive

unless an authoritative source explicitly supports that status.

12. OLD WAY VS GREENLIGHT

Show a short comparison.

The old way

Discover program

- > find the right website
- > determine which institution actually runs it
- > interpret requirements
- > find the correct contact or portal
- > collect documents

- > fill application
- > write email
- > follow up

Greenlight

Get it for me ->

This button is critical.

13. "GET IT FOR ME" WORKFLOW

When clicked, transition into an agent execution view.

Do not show chain-of-thought.

Show observable actions and results.

Example:

Preparing your next step

Program verification

- ✓ Current program requirements loaded

Household matching

- ✓ Province matches
- ✓ Utility type matches
- ✓ Program category matches
- ? Homeowner status unresolved

Contact resolution

- ✓ Program administrator identified
- ✓ Correct action channel determined
- ✓ Official contact verified

Documentation

- ✓ Utility bill attached
- ✓ Provider extracted
- ✓ Account type identified
- ! Proof of ownership missing

Application

- ✓ 9 fields completed
- ? 2 questions require you
- 👉 Final certification requires approval

Communication

- ✓ Recipient verified
- ✓ Email draft prepared

14. CONTACT RESOLUTION & ACTION ROUTING

This is a CORE subsystem.

Greenlight must not merely draft an email.

It must first determine:

Who actually owns the next required action?

Possible destinations:

- provincial ministry
- federal department
- municipality
- local electricity distributor
- gas utility
- program administrator
- energy-efficiency organization
- approved service organization
- contractor
- financing provider
- online application portal
- utility portal
- document upload system
- phone support channel

The funding government, program administrator, utility, and application recipient may be different organizations.

Never assume they are the same.

15. ACTION ROUTING LOGIC

For every opportunity, Greenlight should determine:

```
type ActionRoute = {
  routeType:
    | "email"
    | "web_application"
    | "utility_portal"
    | "government_portal"
    | "phone"
    | "contractor"
    | "service_organization"
    | "document_upload"
    | "manual_review";

  administeringOrganization: string;
  departmentOrProgram?: string;

  contact?: {
    email?: string;
    phone?: string;
    contactName?: string;
  };

  applicationUrl?: string;

  preferredSubmissionMethod?: string;

  sourceUrl: string;
  sourceTitle?: string;
```

```
lastVerifiedAt: string;
confidence: number;
notes?: string[];
};
```

16. VERIFIED CONTACT RULE

Gemini may reason about:

- which organization should be contacted
- which department is likely relevant
- which type of action is required
- what information the user needs to communicate

Gemini must NEVER invent:

- email addresses
- phone numbers
- URLs
- department names
- staff names
- ministry contacts
- utility contacts

All external contact data must come from:

11. a trusted structured database
12. an authoritative retrieved source
13. a verified official program page

If no official contact can be verified:

Display:

No verified contact found

Then:

We found the program, but we could not verify the correct external contact channel.

Never hallucinate a plausible address.

17. CONTACT RESOLUTION TOOL

Implement a tool/service abstraction such as:

```
resolveProgramContact(programId, userContext)
```

It should return structured verified information.

Possible implementation:

```
/lib/actions/
  resolve-program-contact.ts
  resolve-action-route.ts
  verify-contact.ts
```

18. ADMINISTRATOR CHAIN VISUAL

This can be a major demo moment.

When resolving a program, show a compact visual chain:

Government of Ontario

↓

Program

↓

Program administrator

↓

Correct action channel

↓

Verified destination ✓

Example:

Funding authority

Program owner

Administrator

Application method

Verified contact

This visually proves that the AI did more than generate copy.

19. EMAIL DRAFTING

Only draft an email AFTER:

14. the program is verified
15. the administrator is resolved
16. the correct contact is verified
17. the system determines email is the correct action channel

Example UI:

Ready to contact the program administrator

To

verified@email.example

Organization

Program Administrator

Subject

Eligibility inquiry - [Program Name]

Message

Personalized draft based only on verified user information.

CTA options:

Edit

Copy

Approve

For the hackathon, do not send real external emails unless a safe test environment is explicitly configured.

20. EMAIL CONTENT RULES

Email drafts should:

- be concise
- identify the relevant program
- state only verified household facts
- ask focused eligibility questions
- never fabricate account details
- never make legal claims
- clearly identify unresolved questions
- avoid excessive personal data

Example logic:

Based on the published requirements, I appear to meet X and Y criteria. Could you confirm whether Z applies to my situation?

21. WHEN EMAIL IS NOT APPROPRIATE

Greenlight should not draft an email simply because it can.

If the official path is:

- application portal
- provider portal
- contractor quote
- phone line
- service organization
- document upload

then show that route instead.

Example:

Best next step

This program does not require an email.

Applications are submitted through the official program portal.

- ✓ 8 required fields prepared
- ✓ Utility details extracted
- ✓ Supporting documents identified

? 2 fields need you

CTA:

Continue application

22. PROVIDER DETECTION

If a utility bill identifies the provider:

- extract it
- verify the provider
- map it to the user's region
- use it to narrow relevant programs
- use it for contact resolution where appropriate

Example:

Provider detected: Toronto Hydro

Then:

Checking provider-specific opportunities...

Then:

Correct program route found.

23. HUMAN-IN-THE-LOOP

The agent should intentionally stop when critical information is missing.

Example:

Do you own or rent this property?

Buttons:

Own

Rent

After selection:

Eligibility confidence updates.

Example:

78% -> 91%

Then:

One requirement resolved.

This proves the AI does not invent missing context.

24. PERMISSION MODEL

Make user control visible.

Greenlight may

- ✓ analyze documents
- ✓ search program data
- ✓ verify contacts
- ✓ calculate scenarios
- ✓ pre-fill forms
- ✓ draft communications

Greenlight needs approval to

- > send communications
- > submit applications
- > share sensitive documents
- > authorize purchases
- > accept legal declarations
- > sign documents

Never auto-submit government forms in the hackathon version.

25. FINANCIAL NEGOTIATOR

Add a goal input:

What do you want Greenlight to optimize?

Preset chips:

- Save me money
- Find every rebate
- Lowest upfront cost
- Biggest long-term savings
- Reduce energy use

Natural language input:

Save me \$1,000 this year without spending more than \$500 upfront.

Convert to structured constraints.

Example:

```
{
  "objective": "maximize_annual_savings",
  "minimum_target_savings": 1000,
  "max_upfront_cost": 500,
  "comfort_reduction_allowed": false,
  "jurisdiction": "ON",
  "country": "CA"
}
```

Do not rely on Gemini for arithmetic.

Gemini interprets.

TypeScript calculates.

26. NEGOTIATED PLAN VIEW

Show:

Your negotiated plan

Example:

Smart thermostat

Upfront cost: \$X

Potential incentive: \$X

Estimated annual savings: \$X

Utility plan adjustment

Upfront cost: \$0

Estimated annual savings: \$X

Insulation

Net upfront cost: \$X

Estimated annual savings: \$X

Summary:

Budget ceiling

\$500

Estimated upfront cost

\$412

Potential first-year value

\$1,084

Estimated energy savings

X

Estimated emissions reduction

X kg CO2e/year

Money dominates visually.

Carbon is secondary.

27. DESKTOP + MOBILE REQUIREMENT

This product MUST be fully functional on both desktop and mobile.

Desktop is likely the primary judging environment.

Mobile must not be an afterthought.

Do not merely make desktop cards stack vertically and call it responsive.

Design both intentionally.

28. DESKTOP LAYOUT

For large screens:

Use spacious multi-column layouts where appropriate.

Examples:

Opportunities

Left:

main opportunity list

Right:

household summary / value found / filter or context panel

Opportunity detail

Main:

evidence, economics, eligibility

Side:

CTA, status, next action

Agent workflow

Left:

execution timeline

Right:

application state / contact resolution / human input

Use desktop width efficiently.

Do not stretch paragraphs to 1400px.

Use reasonable max-widths.

29. MOBILE LAYOUT

For mobile:

Prioritize one action at a time.

Use:

- sticky bottom CTA where helpful
- collapsible evidence
- full-width cards
- progressive disclosure
- bottom sheets
- compact timelines
- horizontally scrollable metric rows only if necessary

Never shrink desktop UI until it becomes unreadable.

Examples:

Mobile opportunity detail

Top:

potential value

Then:

eligibility

Then:

economics

Then:

evidence accordion

Sticky CTA:

Get it for me

Mobile agent workflow

Top:

current step

Then:

timeline

Then:

human question

Sticky CTA:

Continue

30. RESPONSIVE BREAKPOINT PHILOSOPHY

Use Tailwind breakpoints intentionally.

Design at least for:

- ~375px mobile
- ~430px large mobile
- ~768px tablet
- ~1024px laptop
- ~1440px desktop

Test key flows at each.

No horizontal overflow.

No clipped modals.

No microscopic tables.

31. MOBILE UPLOAD EXPERIENCE

Allow mobile users to:

- choose PDF

- choose image
- take a photo of a utility bill

Use:

```
accept="application/pdf,image/*"
```

and mobile-friendly camera capture where supported.

Document preview must work on mobile.

32. MOBILE NAVIGATION

Keep navigation minimal.

Preferred:

Desktop:

top navigation

Mobile:

simple compact header + optional bottom navigation only if justified

Do not create a complex hamburger menu for four screens.

33. TOUCH INTERACTIONS

All touch targets should be large enough.

Minimum practical target:

~44px height/width where applicable.

No hover-dependent functionality.

Anything important on hover desktop must also work by tap.

34. RESPONSIVE TYPOGRAPHY

Hero numbers like:

\$3,240

must scale.

Use responsive typography.

Desktop:

dramatic

Mobile:

still prominent without wrapping awkwardly

Use `clamp()` or Tailwind responsive classes.

35. MOBILE PERFORMANCE

Do not ship giant animation bundles.

Optimize:

- images
- document previews
- initial JS
- large libraries

Framer Motion should be used selectively.

Mobile transitions should remain smooth.

36. SCREEN ARCHITECTURE

Build routes/screens:

``/``

Landing / upload

``/analyze``

Analysis state

``/opportunities``

Findings

``/opportunity/[id]``

Opportunity detail

``/agent/[caseId]``

Agent execution

``/plan``

Negotiator

``/profile``

Household profile

``/demo``

Deterministic presentation mode

Optional:

``/presentation``

Pitch support screens

37. VISUAL DESIGN DIRECTION

Do not generate generic hackathon UI.

Do not create:

- neon gradients
- AI purple
- giant glowing blobs
- floating glass cards
- particle effects
- glowing node networks
- leaf logos
- earth illustrations
- random green gradients
- generic AI sparkle icons

The visual language should feel closer to:

- premium fintech
- institutional intelligence tools
- Linear
- Stripe
- Arc
- modern Bloomberg-like financial software
- high-end editorial interfaces

Not literal copies.

38. COLOR LANGUAGE

Use restrained colors.

Background:

warm neutral / near-white

Text:

near-black

Secondary:

graphite

Financial success:

restrained green

Warning:

muted amber

Problem:

subtle red

Green should mean something.

Do not make the entire app green.

39. TYPOGRAPHY

Use:

- Geist

- Inter
- Instrument Sans
- equivalent

Numbers should feel premium.

Use tabular numerals where appropriate.

The hero:

\$3,240

must carry visual weight.

40. INFORMATION DESIGN

Avoid endless rounded-card grids.

Use:

- sections
- dividers
- structured rows
- evidence matrices
- timelines
- drawers
- side panels
- expandable rows
- editorial layouts

Not every element needs a box.

41. MOTION

Use Framer Motion only where useful.

Good motion:

- document compresses into sidebar
- value counts upward
- opportunity ordering changes
- confidence updates
- timeline events complete
- application packet assembles
- contact chain resolves

Bad motion:

- floating objects
- infinite pulse
- decorative parallax
- generic AI shimmer everywhere

Respect reduced-motion.

42. ACCESSIBILITY

Implement:

- semantic HTML
- keyboard support
- proper focus states
- ARIA where needed
- sufficient contrast
- reduced-motion support
- accessible form labels
- screen-reader-friendly progress states

43. TECH STACK

Prefer:

Frontend

Next.js App Router

TypeScript

React

Tailwind CSS

Framer Motion

Lucide icons

Use existing repo versions if already installed.

Avoid unnecessary dependency churn.

44. BACKEND

Use:

Next.js server routes / server actions

Supabase for:

- PostgreSQL
- document metadata
- file storage
- auth if needed
- realtime agent state if useful

Never expose service-role secrets client-side.

45. GEMINI ARCHITECTURE

Use the CURRENT recommended Google Gemini developer SDK/API at implementation time.

Check official Google documentation before coding exact syntax.

Use Gemini for:

18. utility-bill extraction
19. multimodal document understanding
20. program language interpretation
21. eligibility reasoning
22. missing-information detection
23. user objective extraction
24. contact-routing reasoning
25. application field mapping
26. communication drafting
27. tool orchestration

Use structured outputs.

46. AI LAYER STRUCTURE

Suggested:

```
/lib/ai/  
  gemini.ts  
  schemas.ts  
  prompts.ts  
  extract-bill.ts  
  match-programs.ts  
  eligibility.ts  
  negotiator.ts  
  action-routing.ts  
  contact-resolution.ts  
  prepare-application.ts  
  draft-communication.ts
```

Do not scatter prompts throughout components.

47. BILL EXTRACTION SCHEMA

```
type UtilityBillExtraction = {  
  provider?: string;  
  
  accountType?:  
    | "residential"  
    | "commercial"  
    | "unknown";  
  
  serviceAddress?: {  
    city?: string;  
    provinceState?: string;  
    postalCode?: string;  
    country?: string;  
  };  
  
  billingPeriod?: {  
    start?: string;  
    end?: string;  
  };  
  
  electricity?: {  
    usageKwh?: number;  
    cost?: number;  
  };  
  
  naturalGas?: {  
    usageM3?: number;  
    usageTherms?: number;  
  };  
};
```



```

    cost?: number;
  };

  totalAmount?: number;
  currency?: string;

  detectedHeatingClues?: string[];

  confidence: number;

  missingCriticalFields: string[];
};

```

Mask account numbers.

Do not persist them unless absolutely required.

48. HOUSEHOLD PROFILE

```

type HouseholdProfile = {
  id: string;
  userId?: string;

  country: string;
  provinceState: string;
  city?: string;
  postalCodePrefix?: string;

  dwellingType?:
    | "detached"
    | "semi_detached"
    | "townhouse"
    | "condo"
    | "apartment"
    | "other";

  tenure?:
    | "owner"
    | "renter"
    | "unknown";

  primaryHeating?:
    | "natural_gas"
    | "electric"
    | "heat_pump"
    | "oil"
    | "propane"
    | "unknown";

  annualElectricityKwh?: number;
  annualGasM3?: number;

  householdSize?: number;

  existingEquipment?: {
    smartThermostat?: boolean;
    heatPump?: boolean;
    solar?: boolean;
    battery?: boolean;
  };

  profileConfidence: number;
};

```

49. INCENTIVE TABLE

```

incentive_programs
- id uuid
- slug text
- name text
- provider text
- administering_organization text
- department_or_program text

```

```

- country text
- province_state text
- municipalities jsonb
- category text
- description text
- incentive_type text
- max_value numeric
- currency text
- active boolean
- start_date date
- end_date date nullable
- official_url text
- last_verified_at timestampz
- source_snapshot jsonb
- created_at timestampz
- updated_at timestampz

```

50. ACTION ROUTES TABLE

Create:

```

program_action_routes
- id uuid
- incentive_id uuid
- route_type text
- administering_organization text
- department_or_program text
- contact_name text nullable
- email text nullable
- phone text nullable
- application_url text nullable
- preferred_submission_method text
- source_url text
- source_title text nullable
- verified boolean
- confidence numeric
- last_verified_at timestampz
- notes jsonb

```

This is essential.

51. ELIGIBILITY RULES

```

eligibility_rules
- id uuid
- incentive_id uuid
- field text
- operator text
- expected_value jsonb
- required boolean
- explanation text
- source_reference text

```

Possible operators:

- equals
- not_equals
- in
- not_in
- greater_than
- less_than
- postal_prefix
- homeowner_required
- equipment_present
- equipment_absent
- utility_provider

- manual_review

52. OPPORTUNITIES

```
opportunities
- id uuid
- household_id uuid
- incentive_id uuid nullable
- category text
- title text
- estimated_incentive numeric
- estimated_upfront_cost numeric
- estimated_annual_savings numeric
- estimated_payback_years numeric
- estimated_co2_reduction numeric
- eligibility_confidence numeric
- status text
- reasoning_summary text
- created_at timestampz
```

53. ELIGIBILITY EVIDENCE

```
eligibility_evidence
- id uuid
- opportunity_id uuid
- criterion text
- observed_value text
- expected_value text
- status text
- source text
- confidence numeric
```

Statuses:

- pass
- fail
- unknown
- manual_review

Judges should be able to inspect why the system thinks someone may qualify.

54. AGENT CASES

```
agent_cases
- id uuid
- household_id uuid
- opportunity_id uuid
- action_route_id uuid nullable
- status text
- progress numeric
- created_at timestampz
- updated_at timestampz
```

55. AGENT EVENTS

```
agent_events
- id uuid
- case_id uuid
- sequence integer
- event_type text
- title text
- description text
```

```
- status text
- metadata jsonb
- requires_human boolean
- created_at timestampz
```

56. APPLICATION FIELDS

```
application_fields
- id uuid
- case_id uuid
- field_key text
- label text
- value text
- source text
- confidence numeric
- requires_user_confirmation boolean
- status text
```

Never treat inferred legal declarations as confirmed.

57. USER GOALS

```
user_goals
- id uuid
- household_id uuid
- raw_prompt text
- objective text
- max_upfront_cost numeric nullable
- minimum_annual_savings numeric nullable
- max_payback_years numeric nullable
- target_emissions_reduction numeric nullable
- comfort_reduction_allowed boolean
- parsed_constraints jsonb
```

58. DEMO DATA

Seed a polished Toronto/Ontario demo household.

Include:

- synthetic utility bill
- sample provider
- household profile
- 4 opportunities
- incentive records
- eligibility rules
- action-route records
- verified-looking but clearly sample fixture contacts where not using live official data
- application fields
- agent events
- drafted communication

Any value that is not sourced from real official data must be clearly marked in code as sample/demo data.

59. LIVE MODE + DEMO MODE

Implement adapters.

Example:

```
interface AnalysisProvider {
  analyzeBill(...): Promise<BillAnalysis>;
}

class GeminiAnalysisProvider implements AnalysisProvider {}
class DemoAnalysisProvider implements AnalysisProvider {}
```

Also create adapters for:

- contact resolution
- opportunity matching
- agent execution
- application preparation

Demo mode must be deterministic.

60. STRUCTURED OUTPUTS

Validate all Gemini outputs with Zod.

Example:

```
const EligibilityAssessmentSchema = z.object({
  status: z.enum([
    "likely_eligible",
    "possibly_eligible",
    "unlikely_eligible",
    "insufficient_information"
  ]),
  confidence: z.number().min(0).max(1),
  satisfiedCriteria: z.array(z.string()),
  missingInformation: z.array(z.string()),
  disqualifiers: z.array(z.string()),
  explanation: z.string()
});
```

If validation fails:

28. retry
29. fail gracefully
30. never inject broken AI output into financial logic

61. TOOL ARCHITECTURE

Potential Gemini tools:

```
searchPrograms()
getProgramDetails()
evaluateDeterministicRules()
getHouseholdProfile()
updateHouseholdProfile()
calculateUpgradeEconomics()
resolveProgramContact()
verifyContactSource()
getRequiredDocuments()
prepareApplicationFields()
draftEmail()
requestUserInput()
```

Do not expose:

```
submitGovernmentApplication()
authorizePurchase()
signLegalDocument()
sendMoney()
```

for the hackathon.

62. CALCULATIONS

All financial arithmetic must be deterministic.

Create:

```
/lib/calculations/  
  annualize-usage.ts  
  payback.ts  
  net-cost.ts  
  roi.ts  
  scenario-ranking.ts  
  constraint-solver.ts
```

Gemini never invents arithmetic.

63. CONSTRAINT SOLVER

Inputs:

- opportunity candidates
- upfront budget
- eligibility confidence
- estimated savings
- compatibility
- exclusions
- estimated incentive
- effort

Goals:

- maximize annual savings
- maximize first-year net value
- maximize lifetime value
- maximize incentive capture
- minimize upfront cost
- reduce energy use

Keep it deterministic.

64. SOURCE TRANSPARENCY

Every program and contact route should show:

- official source
- last verified
- relevant requirement
- action channel
- verification state

One of the product's core principles:

AI interprets. Evidence decides.

65. CONTACT CONFIDENCE

Do not show fake precision.

Use:

- High
- Medium
- Low

or rounded percentages.

Confidence should reflect:

- source authority
- source freshness
- exactness of administrator match
- whether contact was explicit on official page
- whether route was inferred

If inferred, label it.

66. DATA SECURITY

Utility bills may contain:

- names
- addresses
- account numbers
- usage information
- balances

Implement:

- server-side AI calls
- no keys client-side
- private Supabase storage
- RLS if auth
- signed URLs
- account-number masking
- delete-file action
- minimal logging
- no document contents in logs

Use synthetic documents in demo mode.

67. AUTH

Do not block the wow moment with signup.

Demo path must work without auth.

If persistence is added:
use magic link or Google OAuth.

68. APPLICATION PACKET

Create a polished view:

Application ready for review

12 / 14 fields complete

Automatically completed

9

Confirmed by you

3

Still needed

2

CTA:

Review application

The agent must create tangible output.

69. SINGLE DOMINANT ACTION RULE

Every screen should have one obvious next step.

Landing:

Upload bill

Findings:

Explore opportunities

Opportunity:

Get it for me

Agent:

Answer question

Contact:

Review draft

Application:

Review application

Negotiator:

Negotiate

70. FAILURE STATES

Gemini document failure:

We couldn't finish reading this document.

Actions:

Try again

Enter details manually

Contact lookup failure:

We found the program, but could not verify the correct contact channel.

Do not invent contact information.

Program retrieval failure:

fallback to cached records.

71. KEY COPY

Landing:

What are you leaving on the table?

Upload a utility bill. Greenlight finds relevant incentives, calculates the economics, figures out who handles the next step, and prepares the work required to pursue it.

Findings:

We found \$3,240 in potential value.

4 programs and efficiency opportunities appear worth investigating.

Agent:

Let Greenlight handle the paperwork.

We'll verify the rules, figure out who actually owns the next step, organize the documents, and prepare everything we can before we need you.

Negotiator:

Give us the constraint.

Placeholder:

Save me \$1,000 this year without spending more than \$500 upfront.

72. WOW MOMENT 1

Animate:

\$0
\$420
\$1,260
\$2,080
\$3,240

Then:

\$3,240 FOUND

73. WOW MOMENT 2

Agent execution:

Program requirements loaded	✓
7 eligibility criteria checked	✓
Program administrator resolved	✓
Official contact channel verified	✓
Supporting documents identified	✓
9 application fields completed	✓
Email draft prepared	✓
2 questions need you	->

Then:

2 questions between you and the next step.

74. WOW MOMENT 3

Contact-resolution chain:

Province
↓
Program
↓
Administrator
↓
Correct department
↓
Verified action route

Then:

Verified destination found.

This must feel like a genuine retrieval/action-routing step.

75. WOW MOMENT 4

User enters:

Save me \$1,000 without spending more than \$500.

UI recalculates.

Then:

Constraint satisfied.

Potential upfront spend

\$412

Potential first-year value

\$1,084

76. PRESENTATION MODE

Add:

/demo

or:

?presentation=true

Presentation mode:

- deterministic fixture
- no auth
- fast animations
- reset button
- large controls
- no external send actions
- stable state
- responsive on laptop and mobile

Optional keyboard shortcut:

R resets demo.

77. OPTIONAL JUDGE BILL MODE

Allow live upload.

Before upload:

You can obscure your name, full address, and account number. We only need the service and usage information.

Mask sensitive identifiers automatically if possible.

78. MOBILE PRESENTATION MODE

The demo should also be runnable from a phone.

Test:

- upload from camera
- bill image preview
- analysis sequence
- findings
- Get it for me

- contact resolution
- human question
- application packet

Do not rely on desktop-only hover states.

79. COMPONENTS

Suggested:

```
BillUploader
DocumentPreview
AnalysisSequence
ValueFoundHero
OpportunityRow
OpportunityDetails
EligibilityMatrix
FinancialBreakdown
EvidenceDrawer
ActionRouteCard
AdministratorChain
VerifiedContactCard
AgentTimeline
AgentEvent
HumanInputCard
PermissionsPanel
ApplicationProgress
ApplicationPacket
GoalInput
ConstraintChips
NegotiatedPlan
Metric
SourceBadge
ConfidenceIndicator
MobileStickyCTA
DemoModeBadge
```

80. FILE STRUCTURE

```
app/
  page.tsx
  analyze/
    opportunities/
    opportunity/[id]/
    agent/[caseId]/
  plan/
  profile/
  demo/
  presentation/

components/

lib/
  ai/
  calculations/
  eligibility/
  actions/
  contacts/
  data/
  demo/
  supabase/
  security/

types/

supabase/
  migrations/
  seed.sql

public/
  demo/
```

```
README.md
DEVPOST.md
.env.example
```

81. API ROUTES

Potential:

```
POST /api/documents/analyze
POST /api/households
POST /api/opportunities/match
POST /api/opportunities/[id]/evaluate
POST /api/actions/resolve
POST /api/contacts/verify
POST /api/agent/cases
POST /api/agent/cases/[id]/continue
POST /api/agent/cases/[id]/answer
POST /api/negotiate
POST /api/communications/draft
```

82. ENVIRONMENT VARIABLES

```
GEMINI_API_KEY=
NEXT_PUBLIC_SUPABASE_URL=
NEXT_PUBLIC_SUPABASE_ANON_KEY=
SUPABASE_SERVICE_ROLE_KEY=
NEXT_PUBLIC_DEMO_MODE=true
```

Never commit secrets.

83. README

Write a strong README covering:

- what Greenlight is
- problem thesis
- architecture
- Gemini usage
- contact-resolution architecture
- data model
- demo mode
- desktop/mobile support
- safety model
- installation
- environment setup
- future roadmap
- hackathon alignment

84. DEVPOST

Create DEVPOST.md.

Sections:

Inspiration

What it does

How we built it

How Gemini is used

How verified contact routing works

Challenges

Accomplishments

What we learned

What's next

85. TESTS

Prioritize:

- financial calculations
- constraint solver
- eligibility rule engine
- action route selection
- source verification behavior
- schema validation

86. PO MUST-WORK PATH

If time becomes constrained, prioritize:

31. polished responsive landing
32. upload/demo bill
33. Gemini bill extraction
34. household profile
35. seeded incentive database
36. matching
37. value reveal
38. opportunity detail
39. Get it for me
40. action routing
41. administrator resolution
42. verified contact display
43. agent timeline
44. one human question
45. application prep
46. email draft OR correct portal route
47. negotiator constraint flow
48. polished desktop + mobile layouts

87. P1

After P0:

- auth
- deeper evidence drawer
- more programs
- live contact retrieval
- real Supabase storage
- multiple household profiles
- responsive polish
- real email draft export

88. P2

Only if core demo is perfect:

- realtime data refresh
- contractor matching
- advanced optimization
- multiple jurisdiction support
- emissions modeling
- actual application integrations
- safe outbound communications

89. DO NOT WASTE TIME ON

- admin dashboard
- subscriptions
- social features
- carbon gamification
- blog
- onboarding carousel
- full CMS
- generic chatbot
- random map
- complex dark mode
- fake testimonials
- fake partner logos

90. DESIGN ANTI-SLOP RULE

Before adding any decorative element, ask:

Does this help explain money, eligibility, evidence, routing, or action?

If not:

remove it.

No:

- glowing orb
- AI sparkle
- green planet
- floating leaves
- neon gradient
- fake graph
- abstract nodes

91. COPY STYLE

Tone:

- confident
- intelligent
- concise
- provocative
- financially literate
- human

Avoid:

- revolutionize
- leverage AI
- unlock
- empower
- seamless
- sustainable future
- make an impact

Preferred:

We found...

Here's what matters.

This appears worth pursuing.

Two questions remain.

We found the right administrator.

Verified contact found.

Let Greenlight handle the paperwork.

92. PRODUCT PRINCIPLES

Keep these in the repo documentation:

1. Money first. Carbon second.
2. Evidence over AI confidence.
3. Search is not action.
4. AI interprets. Code calculates.
5. Verified routing before communication.
6. Never invent an external contact.
7. Human approval at consequential boundaries.
8. Never make the user decode bureaucracy.
9. One screen, one obvious next action.
10. Mobile is not a reduced desktop. Design both intentionally.

93. FINAL DEMO FLOW

Judge sees utility bill.

↓

Gemini understands it.

↓

Household profile appears.

↓

\$3,240 found.

↓

Judge thinks:

"Okay, rebate finder."

↓

Click:

Get it for me.

↓

Greenlight:

- verifies program
- checks eligibility
- determines who administers it
- finds the correct action path
- verifies official contact information
- identifies documents
- pre-fills application fields
- drafts the communication

↓

System stops:

Do you own or rent this property?

Judge chooses:

Own

↓

Eligibility improves.

↓

Administrator chain resolves.

↓

Verified destination found.

↓

Draft email or application route appears.

↓

Then user enters:

Save me \$1,000 without spending more than \$500.

↓

Plan recalculates.

↓

Constraint satisfied.

↓

Reveal:

This is not a carbon tracker.

This is not a rebate finder.

This is an economic agent for the green transition.

94. FINAL IMPLEMENTATION BEHAVIOR

Before writing code:

49. inspect repository
50. inspect package versions
51. inspect existing design system
52. inspect environment variables
53. identify reusable components
54. verify current official Gemini SDK/API patterns
55. verify current Supabase patterns if used
56. determine responsive architecture
57. determine demo adapter strategy

Then build.

Do not repeatedly ask me implementation questions.

Make strong reasonable decisions.

If external integration is unreliable:

use an adapter and deterministic fallback.

If contact data cannot be verified:

show that honestly.

If a live program is unavailable:

use a sample fixture.

95. SELF-REVIEW BEFORE FINISHING

Evaluate:

Product

Does this feel like an agent, not a rebate directory?

Action

Does the system actually route the user toward completion?

Contact Resolution

Does it find the correct institution instead of inventing one?

Gemini

Is Gemini doing real multimodal/reasoning/tool work?

Design

Does it look premium?

Mobile

Can every critical path be completed comfortably on a phone?

Desktop

Does it use space intelligently?

Trust

Can users inspect sources?

Demo

Can someone understand it in 15 seconds?

Reliability

Does demo mode work without external dependencies?

Fix weaknesses before declaring completion.

96. FINAL POLISH CHECKLIST

Before completion:

- zero console errors
- no broken responsive states
- no horizontal mobile overflow
- touch targets tested
- upload works mobile and desktop
- no hover-only critical interactions
- demo reset works
- financial math verified
- action routing works
- contact verification clearly labeled
- unverified contact never displayed as verified
- no real secrets committed
- Gemini failure states work
- sample data labeled internally
- loading states polished
- accessibility checked
- performance reasonable on mobile
- README complete
- Devpost draft complete

97. PHILOSOPHICAL END STATE

Today's climate software often asks:

How can we make people care more?

Greenlight asks:

How can we make caring less necessary?

If the sustainable decision becomes:

- cheaper
- easier
- administratively simpler
- financially rational
- automatically routed to the right institution

then adoption no longer depends entirely on altruism.

That is the thesis.

98. MEMORABLE LINES

Use selectively:

Search is not action.

A rebate finder gives you a link. Greenlight gives you an agent.

The green transition has a UX problem.

The money already exists. The missing layer is execution.

We don't just find the program. We figure out who actually handles it.

If someone is willing to pay you to go green, your AI should make sure you collect.

99. NOW BUILD IT

Start by inspecting the repository.

Then build the strongest P0 implementation possible.

Prioritize:

demo quality > feature quantity

working agent behavior > fake complexity

verified routing > impressive-looking hallucinations

desktop + mobile quality > desktop-only polish

evidence > AI claims

action > information

Do not stop after scaffolding.

Do not turn the app into a chatbot.

Do not generate generic AI visuals.

Build the product so that the first 90 seconds of the demo tell the entire story without needing us to explain every screen.